

QWEN

Helix Constitution — Universal QWEN.md

Field	Value
Revision	2
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Mirrored Constitution.md §11.4.78 (CodeGraph code-intelligence mandate) into this QWEN.md. Revision 1 created the file per user mandate 2026-05-20 carrying the §11.4.76 + §11.4.77 mirrors.
Issues	none
Issues summary	—
Fixed	§11.4.78 mirror
Fixed summary	§11.4.78 lands in lockstep with the Constitution.md §11.4.78 addition.
Continuation	—

Table of contents

- [How inheritance works](#)
- [Identity & posture](#)
- [Top-level invariants for every agent](#)
- [Critical base rules restated \(for agents that don't honour @imports\)](#)
 - [§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE](#)
 - [§1.1 — Mutation-paired gates](#)

- [§11.4.10 — Credentials-handling mandate](#)
- [§11.4.17 — Universal-vs-project classification](#)
- [§11.4.20 — Subagent-driven-by-default](#)
- [§11.4.73 — Main-specification document versioning + revision discipline](#)
- [§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement](#)
- [§11.4.76 — Containers-submodule mandate](#)
- [Companion documents](#)

How inheritance works

This QWEN.md is for the **Qwen Code CLI agent** (qwen-code). It documents the universal Helix Constitution from the Qwen agent's perspective — the same content seen by Claude Code via CLAUDE.md and by OpenCode / Cursor / generic AI tooling via AGENTS.md.

The **authoritative source** is Constitution.md in this repository. When this file diverges from Constitution.md, the latter wins. This file exists because:

1. Qwen Code does not always honor @import directives across files; restating the critical rules inline ensures the agent reads them on every session.
2. Cross-agent parity: Claude Code (CLAUDE.md), OpenCode/ Cursor (AGENTS.md), Qwen Code (QWEN.md) all start from the same constitutional baseline.

Discover this file via the canonical parent-walk pattern documented in every consuming project's CLAUDE.md / AGENTS.md:

```
find_constitution_root() {
    local cur="${1:-$PWD}"
    while [[ "${cur}" != "/" ]]; do
        if [[ -f "${cur}/constitution/Constitution.md" ]]; then
            echo "${cur}/constitution"; return 0
        fi
        cur="$(dirname "${cur}")"
    done
    return 1
}
```

Identity & posture

When the Qwen Code agent loads this file as part of session bootstrap, it operates under the Helix Constitution with the same identity, anti-bluff posture, and end-user quality covenant as any other CLI agent in the catalogue. There is no "Qwen-lite" interpretation — Qwen Code is held to the full §11.4 covenant.

Top-level invariants for every agent

1. **Read order on a cold start.** CLAUDE.md / AGENTS.md / QWEN.md (this file) for the AI-agent posture; Constitution.md for the canonical universal rules; then the consuming project's CLAUDE.md for project-specific extensions.
2. **Multi-mirror push.** Every commit on this submodule MUST land on all four canonical mirrors (GitHub + GitLab + GitFlic + GitVerse) per §2.1.
3. **Anti-bluff is non-negotiable.** Passing tests MUST imply working features. Bluffing PASSes (and FAILs that hide root cause) is a release blocker (§11.4 + §11.4.1).
4. **Submodule-catalogue-first.** Before scaffolding anything, survey the vasic-digital + HelixDevelopment orgs for an existing Submodule (§11.4.74). Reuse → extend → no-match in that order.
5. **Containers-submodule for ALL container workloads.** Use vasic-digital/containers (§11.4.76) — never reinvent compose/boot orchestration in-project.

Critical base rules restated (for agents that don't honour @imports)

§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule. Captured tests MUST exercise the behavior they claim to verify. PASS-without-execution paths, mock-only tests that don't round-trip, skip-by-default integration tests that mask real failures, metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-without-runtime PASS are all critical defects regardless of how green the summary line looks. Each test failure in CI MUST imply a real broken feature; each PASS MUST imply the feature actually works for the end user. Tests and HelixQA Challenges are bound EQUALLY.

The verbatim covenant **MUST** be present in every consumer governance file (project Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md) and in every owned submodule's equivalent. Tools that don't expand @imports still read the literal text.

§1.1 — Mutation-paired gates

Every captured-evidence gate **MUST** ship with a paired mutation test that mutates the gate's anchor + runs the gate + asserts the gate now FAILs. Absence of the paired mutation is itself a bluff.

§11.4.10 — Credentials-handling mandate

Credentials are **NEVER** committed to git, **NEVER** logged, **NEVER** printed to stdout/stderr. .env files are .gitignored; committed siblings are .env.example placeholders only. Pre-store leak audit (§11.4.10.A) scans every PR for credential patterns before merge.

§11.4.17 — Universal-vs-project classification

Every new rule **MUST** declare itself **universal** (applies to every project consuming this Constitution) or **project** (applies only to one specific project). Misclassification (universal rule landing in a project's local CLAUDE.md instead of the constitution submodule) is a release blocker.

§11.4.20 — Subagent-driven-by-default

When a CLI agent (Qwen Code included) has a subagent / Task tool available, the default mode of operation is to dispatch subagents for discrete units of work rather than perform them inline. This protects context, enables parallel work, and matches the §11.4.27 no-fakes-beyond-unit-tests posture (each subagent runs real tools).

§11.4.73 — Main-specification document versioning + revision discipline

Projects with a main specification (docs/specs/.../specification.V<N>.md-shaped) maintain **TWO** version axes:

- **Primary** (V1 / V2 / V3 ...) bumps for major rewrites only.
- **Secondary** (Revision N in the metadata table) bumps for additive changes within a primary version.

Spec edits trigger mandatory comprehensive planning and implementation in the consuming project — they are not isolated doc tweaks (the "spec ripple" rule).

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement

Direct user mandate (verbatim, 2026-05-20): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the Qwen Code agent MUST: (1) survey vasic-digital + HelixDevelopment on GitHub + GitLab — the canonical inventory is submodules-catalogue.md (142 repos categorised); (2) reuse an existing Submodule when it covers $\geq 80\%$; (3) extend in-place via upstream PR when $80\%+$ matches; (4) document the survey result via Catalogue-Check: `reuse|extend|no-match <org/repo>@<sha>` in the tracker row.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

§11.4.76 — Containers-submodule mandate

Direct user mandate (verbatim, 2026-05-20): "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: <https://github.com/vasic-digital/containers> (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containerizing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the Qwen Code agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule's `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman` machine / `docker compose` up manually — the boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing —

never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components **MUST** actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test **MUST** imply the infra was up.

Tracker rows touching containerization **MUST** record Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse); no-match requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate CM-CONTAINERS-USED scans container-touching PRs for digital.vasic.containers/... imports. Paired mutation strips the import + asserts FAIL.

Canonical authority: Constitution.md §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate

Direct user mandate (verbatim, 2026-05-20): "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule!"

Forensic anchor. 2026-05-20T15:00Z: ATMOSphere parent's audio Tier 1 commit_all.sh stalled 4 h on git add -A scanning 274 GiB .git-backup-* + 159 GiB RKTools/linux/ + 167 GiB qa-results/ — all untracked but un-gitignored. Bare .gitignore fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every .gitignore entry the Qwen Code agent considers adding that excludes (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project **MUST** carry a documented + automated mechanism to either **re-obtain** (vendor tarball, SDK installer, package registry, git submodule, object store) OR **re-generate** (build pipeline, code-gen, asset render, captured-evidence replay, container build).

Required artefacts per qualifying .gitignore entry: (1) .gitignore-meta/<entry-slug>.yaml declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) post-clone bootstrap entry (scripts/setup.sh or canonical equivalent) running the mechanism non-interactively; (3) pre-build gate

verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok recent`; (4) README + docs/guides/*.md describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are §11.4 PASS-bluff variants — codebase appears complete but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate CM-GITIGNORE-REGEN-MECHANISM scans every `.gitignore` addition for matching `.gitignore-meta/` sibling. Paired §1.1 mutation strips a required YAML key → gate FAILs.

Composes with §11.4.6, §11.4.65, §11.4.66, §11.4.71, §11.4.74, §11.4.75, §11.4.76, §9 / §9.2, §3.

Canonical authority: Constitution.md §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate

Direct user mandate (verbatim, 2026-05-20): "Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

Every consuming project worked on by AI coding agents — Qwen Code included — MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph>, npm @colbymchenry/codegraph): a local SQLite semantic code-knowledge-graph exposed to agents over MCP, 100% local. Install globally via npm (no sudo). `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the exclude list MUST exclude other-owned submodules and every §11.4.10 credential/secret path. Wire the `codegraph serve --mcp` server into every CLI agent the developers use — for Qwen Code via `.qwen/settings.json` (`qwen mcp add codegraph codegraph serve --mcp --scope project --transport stdio`). Cover the integration with an anti-bluff verification suite using an unforgeable per-agent challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps, never faked PASSes. Document in `docs/CODEGRAPH.md`. CodeGraph is consumed as the npm package (§11.4.74), not a git submodule.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4, §1.1.

Canonical authority: Constitution.md §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index

Direct user mandate (verbatim, 2026-05-21): "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

Refines §11.4.78 step 2 with a per-submodule-ownership split. **Own-org submodules** — full write access via the project's CLIs (canonical orgs: vasic-digital + HelixDevelopment) — MUST be INCLUDED in the index so Qwen Code (and every other AI agent) sees a unified call-graph across the project's domain + own-org infra code. **Third-party submodules** (the §11.4.74 no-match → vendor path) MUST be EXCLUDED. Operational steps: (1) git submodule update --remote --merge to pull latest before re-indexing — respect load-bearing third-party pins. (2) Adjust .codegraph/config.json exclude list. (3) Re-index via scripts/codegraph_setup.sh. (4) Validate via scripts/codegraph_validate.sh probe resolving a symbol that lives ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add own-org path to exclude → validate FAILs → restore.

Composes with §11.4.74, §11.4.78, §11.4.10, §1.1, §107.

Canonical authority: [Constitution.md](#) §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded without an audit trail in .codegraph/config.json) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate

Direct user mandate (verbatim, 2026-05-21): "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed [...] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule [...] Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents [...]"

Three deliverables in this constitution submodule: (1) scripts/codegraph_update.sh — npm-installs latest; §107 anti-bluff verifies codegraph --version reflects the new version. (2) scripts/

codegraph_sync.sh — runs status → sync → status → project's validate; appends to both project's + constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md ledgers, exported per §11.4.65 to .html + .pdf.

Cadence: weekly floor (§11.4.45). Scripts inherited by reference (§3) — Qwen Code consuming projects invoke them at \${CONST_DIR}/scripts/codegraph_*.sh, never copy.

Composes with §11.4.78, §11.4.79, §11.4.10, §11.4.45, §11.4.65, §107, §1.1.

Canonical authority: Constitution.md §11.4.80.

Non-compliance is a process violation; severe cases (>2 weeks since last update + open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

Every consuming project whose supported-platforms manifest lists more than one OS MUST ship per-OS-equivalent implementations + tests for every feature/gate/challenge/mutation that depends on platform-specific primitives. Runtime dispatch via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A)** Per-OS implementation REQUIRED — `cgroup/systemd//proc` primitives have documented equivalents (POSIX `setrlimit/ulimit`, `launchd`, `rctl`, Job Object). **(B)** Per-OS tests REQUIRED — every gate test has case "`$(uname -s)`" in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason only when the platform genuinely cannot enforce. **(C)** Honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes → SKIP with exact reproducer + adjacent test of what IS enforced, e.g. `RLIMIT_CPU+SIGXCPU`). The adjacent test is itself anti-bluff per §11.4 with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical): `systemd-run --user --scope ↔ POSIX ulimit -t -u / launchd; cgroup MemoryMax ↔ XNU gap (use RLIMIT_CPU adjacent) / rctl / Job Object; cgroup TasksMax ↔ RLIMIT_NPROC; /proc/<pid>/oom_score_adj ↔ no Darwin/BSD equivalent.`

Composes with §11.4.1 / §11.4.2 / §11.4.3 (strictened — SKIP only when kernel cannot) / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.27 / §11.4.69 / §11.4.70 / §107. Pre-build gate CM-CROSS-PLATFORM-PARITY + paired §1.1 mutation. No escape hatch.

Canonical authority: [Constitution.md](#) §11.4.81.

Non-compliance is a release blocker on multi-platform projects.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): "How can we speed-up this whole development and fixing process? ... all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md!"

Iteration cycle time bounds the project's defect-discovery rate. Slow cycles ship fewer validated features per unit of operator time. The 2026-05-22 forensic session witnessed ~3 hours of operator wait on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following 9 speedup disciplines AS MANDATORY:

- **(A) Phase 1 forensic before any speculative source patch.** Speculative patches without root-cause evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any re-build.** §11.4.51 strengthened to release-blocker. Skipping live-probe for LIVE_ADB_TESTABLE changes = §11.4.82 violation.
- **(C) Pre-flight before rebuild orchestrators.** 30-sec readiness check verifies device + sink reachability, memory budget, lock files, orphan processes.
- **(D) Persistent build caches outside containers.** ccache + Soong / sccache / Gradle daemon state bind-mounted to host. Subsequent rebuilds drop from ~40 min to 5-15 min.
- **(E) Module-only rebuild for CONFIG_*=m driver patches.** make modules on single driver dir saves 15-20 min/cycle when applicable.
- **(F) Parallel multi-device testing.** Every owned validation device runs the autonomous cycle concurrently with separate output dirs. Catches per-device defects one cycle earlier.
- **(G) Subagent scope ≤30 min + worktree isolation + single-responsibility.** Empirical pattern: 8-task subagents stall, 2-3-task subagents complete.
- **(H) Lock-file + stale-process hygiene.** Clean .git/index.lock + orphan processes on session start. Disable auto git-gc in concurrent multi-agent repos.

- **(I) Cycle telemetry per §11.4.24.** Every cycle logs commit, per-phase wall-clock, speedup flag set, outcome. Weekly aggregation surfaces empirical ROI per discipline.

Composes with §11.4.4 / §11.4.6 / §11.4.9 / §11.4.20 / §11.4.24 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.51 / §11.4.52 / §11.4.58 / §11.4.70 / §12.7 / §107. Pre-build gate CM-ITERATION-SPEEDUP-DISCIPLINE (when implemented) + paired §1.1 mutation. No escape hatch — no --skip-phase1, --no-pre-flight, --rebuild-everything, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag exists.

Canonical authority: Constitution.md §11.4.82.

Non-compliance is a release blocker.

Companion documents

- Constitution.md — authoritative universal Constitution. ALWAYS the tie-breaker.
- CLAUDE.md — Claude Code agent's view of the universal constitution.
- AGENTS.md — OpenCode / Cursor / generic-tooling view.
- README.md — high-level overview + multi-mirror contract.

When operating in a consuming project, ALSO read the project's local QWEN.md / CLAUDE.md / AGENTS.md for project-specific extensions; these MUST NOT weaken any universal rule but MAY add stricter project-specific constraints.